

Assignment 2

INTE2402/2691 Cloud Security

Q1. Cloud Data Checksum with SHA-3

AWS S3 uses checksum mechanisms to ensure data integrity during upload and download.

To compute the SHA-3 checksum first needs exactly 200 bits of data which translates to exactly 25 ASCII characters.

The 25 characters string: ***Anthony Fernando S4129857***

When we convert this string into their ASCII Hexadecimal equivalents and arrange them row by row into the 5 x 5 state matrix,

```
41 6E 74 68 6F
6E 79 20 46 65
72 6E 61 6E 64
6F 20 53 34 31
32 39 38 35 37
```

(1) What is the output of θ mapping in the first round?

The θ mapping provides diffusion. It calculates the parity using XOR of every 5-byte column. Then, for every individual byte in the matrix, it XORs that byte with the parity of the column immediately to its left, and the *rotated* parity of the column immediately to its right. Result;

```
E9 D2 96 E6 2E
C6 C5 C2 C8 24
DA D2 83 E0 25
C7 9C B1 BA 70
9A 85 DA BB 76
```

(2) What is the output of ρ mapping in the first round?

The ρ mapping provides dispersion by rotating the bits within each individual lane. Every (x, y) coordinate has a predefined rotation constant defined by the Keccak specification. We apply a circular left shift to each byte based on its specific constant (We use modulo 8 as there are 8 bits).

Result;

```
E9 A5 A5 6E 71
6C 5C B0 64 42
D6 4B 1C C1 92
8F 93 D8 57 70
6A 16 5B BB 76
```

(3) What is the output of π mapping in the first round?

The π mapping rearranges the physical location of the bytes across the matrix, disrupting vertical alignments. It uses the formula $(x, y) \rightarrow ((x + 3y) \pmod{5}, x)$. This means a byte sitting at a specific coordinate is picked up and placed at a entirely new coordinate in the grid, to shuffle the matrix.

Result;

```
E9 5C 1C 57 76
6E 42 D6 93 5B
A5 B0 C1 70 6A
71 6C 4B D8 BB
A5 64 92 8F 16
```

(4) What is the output of χ mapping in the first round?

The χ mapping introduces non-linearity. For each byte, it looks at the two bytes immediately to its right in the same row. It performs a bitwise NOT on the first adjacent byte, applies a bitwise AND with the second adjacent byte, and then XORs that result into the original byte. Result;

```
E9 1F 3C DE 62
FA 43 9E B7 5B
E4 80 CB F5 7A
72 FC 68 98 B7
37 69 82 2E 56
```

(5) What is the output of ι mapping in the first round?

The ι mapping disrupts any remaining symmetries by modifying only the very first byte of the matrix at coordinate (0,0). It XORs this byte with a predefined "Round Constant." Because this is the very first round of the algorithm, the Round Constant is simply 0x01.

```
E8 1F 3C DE 62
FA 43 9E B7 5B
E4 80 CB F5 7A
72 FC 68 98 B7
37 69 82 2E 56
```

---Initial State Output---

41 6E 74 68 6F
6E 79 20 46 65
72 6E 61 6E 64
6F 20 53 34 31
32 39 38 35 37

---Theta Output---

E9 D2 96 E6 2E
C6 C5 C2 C8 24
DA D2 83 E0 25
C7 9C B1 BA 70
9A 85 DA BB 76

---Rho Output---

E9 A5 A5 6E 71
6C 5C B0 64 42
D6 4B 1C C1 92
8F 93 D8 57 70
6A 16 5B BB 76

---Pi Output---

E9 5C 1C 57 76
6E 42 D6 93 5B
A5 B0 C1 70 6A
71 6C 4B D8 BB
A5 64 92 8F 16

---Chi Output---

E9 1F 3C DE 62
FA 43 9E B7 5B
E4 80 CB F5 7A
72 FC 68 98 B7
37 69 82 2E 56

---Iota Output---

E8 1F 3C DE 62
FA 43 9E B7 5B
E4 80 CB F5 7A
72 FC 68 98 B7
37 69 82 2E 56

Q2. Signing AWS Requests with Signature Version 4

AWS Signature Version 4 (SigV4) is the industry standard for authenticating API requests to AWS. There we have a process called “Key Derivation” which use the master key to generate a temporary key that is scoped to a specific date, region, and service.

```
-- PYTHON: C:/Users/13hankumar/Desktop/Downloads/Assignment_1/q2_aws_sigv4.py --
--- Intermediate Keys (Shown in Hex for Assignment formatting) ---
(1) kDate:      3e55224c50ffbbef0f38567379702cc28dd5cc322f22470f57100e8a20ea23d1
(2) kRegion:    8899a371cb200a61bb1f1d3f5533f2f381c84424f29f5d37d624f71e91699bc8
(3) kService:   11622d012db72de62dea17db9d6fbfff902ae731a78d4e0f0a395e34af30e714
(4) kSigning:    97314cbe3f9d8e95cd4d7e21a9a85314ba2640edac75ede09a6ea0771a0065dc

--- Final Result ---
(5) Signature:  349fb38d1103fc17511478eb897c8114681177f3544042f65bd9e0ecf8924da4
```

(1) Compute $kDate = \text{HMAC}(\text{"AWS4"} + kSecret, \text{Date})$, where $\text{Date} = 20260414$;

To start the key derivation, AWS requires us to prepend the string "AWS4" to our master secret key. We use this new string as the cryptographic key, and the Date as the message for HMAC-SHA256 function. This ensures the derived key is only valid for this specific day.

Key- **AWS4S4129857/K7MDENG+bPxRfiCYEXAMPLEKEY**

Message- **20260414**

Result- **3e55224c50ffbbef0f38567379702cc28dd5cc322f22470f57100e8a20ea23d1**

(2) Compute $kRegion = \text{HMAC}(kDate, \text{Region})$, where $\text{Region} = \text{us-east-1}$;

Next take the raw binary output from Step 1- $kDate$ and use it as the new cryptographic key. The message is the Region. This scopes our key down so it can only be used in the US East 1 region.

Key- **$kDate$**

Message- **us-east-1**

Result- **8899a371cb200a61bb1f1d3f5533f2f381c84424f29f5d37d624f71e91699bc8**

(3) Compute $kService = \text{HMAC}(kRegion, \text{Service})$, where $\text{Service} = \text{iam}$;

Then use the output from Step 2- $kRegion$ as the key, and the Service as the message. Now the key is restricted entirely to the IAM service.

Key- **$kRegion$**

Message- **iam**

Result- **11622d012db72de62dea17db9d6fbfff902ae731a78d4e0f0a395e34af30e714**

(4) Compute $kSigning = \text{HMAC}(kService, \text{"aws4_request"})$;

Then use the output from Step 3-kService as the key, and a special termination string "aws4_request" as the message. The output of this step is our final, fully-scoped signing key.

Key- **kService**

Message- **aws4_request**

Result- **97314cbe3f9d8e95cd4d7e21a9a85314ba2640edac75ede09a6ea0771a0065dc**

(5) Compute the signature = HexEncode(HMAC(kSigning, string to sign))

Now we have specific kSigning key, we use it to hash the entire multi-line "String to Sign". Until this point, all intermediate keys were kept in raw binary format. For this final step, we encode the binary output into a Hexadecimal string. This hex string is the final signature that gets attached to the HTTP request.

Result Signature- **349fb38d1103fc17511478eb897c8114681177f3544042f65bd9e0ecf8924da4**

Q3. AWS Kerberos Authentication

AWS Kerberos authentication can be used to securely verify users when they connect to a MySQL database instance.

AES-128 in CBC (Cipher Block Chaining) mode is implemented. For that keys from MD5 keys are used. MD5 keys are generated using my name and student ID.

(1) Implement AES-128 encryption and decryption algorithms in CBC mode.

Before the protocol begins, symmetric keys must be established. In this implementation, the master keys are derived using the MD5 hash function to generate 128-bit keys suitable for AES-128 encryption.

Client Key (K_c)- Generated by hashing the client's first name concatenated with the student ID. This key is known only to the Client and the Authentication Server (AS).

Server Key (K_s)- Generated by hashing the surname designation concatenated with the student ID. This key is known only to the target Server and the AS.

Session Key ($K_{c,s}$)- A randomly generated 128-bit key created by the AS to facilitate secure communication exclusively between the Client and the Server for this specific session.

(2) Assume C=your first name, S=your surname, K_c =MD5(C, your student ID), K_s =MD5(S, your student ID), Lt=8 hours, n_c =MD5(C),

(i) implement each step of Phase 1 of Simplified Kerberos using AES-128 in (1)

Phase 1 is to securely distribute the new Session Key $K_{c,s}$ to both the Client and the Server without sending it in plaintext. This phase has 2 steps as ticket generation and AS reply to client.

(ii) implement each step of Phase 2 of Simplified Kerberos using AES-128 in (1)

Phase 2 is to authenticate Client and Server mutually each other using the Session Key provided by the AS. In this phase client generates Authenticator and send it to server, then server will reply to it.

(3) What is the ticket (Hex numbers) in your implementation? What is the authenticator (Hex numbers) in your implementation?

```
=== RESTART: C:\Users\IshanKumarasinghe\Downloads\Assignment_1\q3_kerberos.py ==
Phase 1- Authentication Server (AS) Exchange
Ticket (Hex)- 50d76bb29f4932a7c315a6abf77bf2146a980dba9a7141ff05d6c411e868a139a1
c2d5d2d42768c872d131d38dda74c1

Phase 2- Client/Server Exchange
Authenticator (Hex)- 3745f585778edc14347663a87acc6073d974866939720fd08f5d30eca34
58a38
```

Ticket (Hex) -

50d76bb29f4932a7c315a6abf77bf2146a980dba9a7141ff05d6c411e868a139a1c2d5d2d42768c87
2d131d38dda74c1

Authenticator (Hex) -

3745f585778edc14347663a87acc6073d974866939720fd08f5d30eca3458a38

(4) Please explain how the user can be authenticated by the server and how the server can be authenticated by the user in the protocol.

Kerberos achieves two way trust.

How the Server Authenticates the User:

The server relies on the trust it places in the Authentication Server (AS). The Ticket provided by the user in Phase 2 is encrypted using the Server's master secret key K_s , which is known *only* to the Server and the AS.

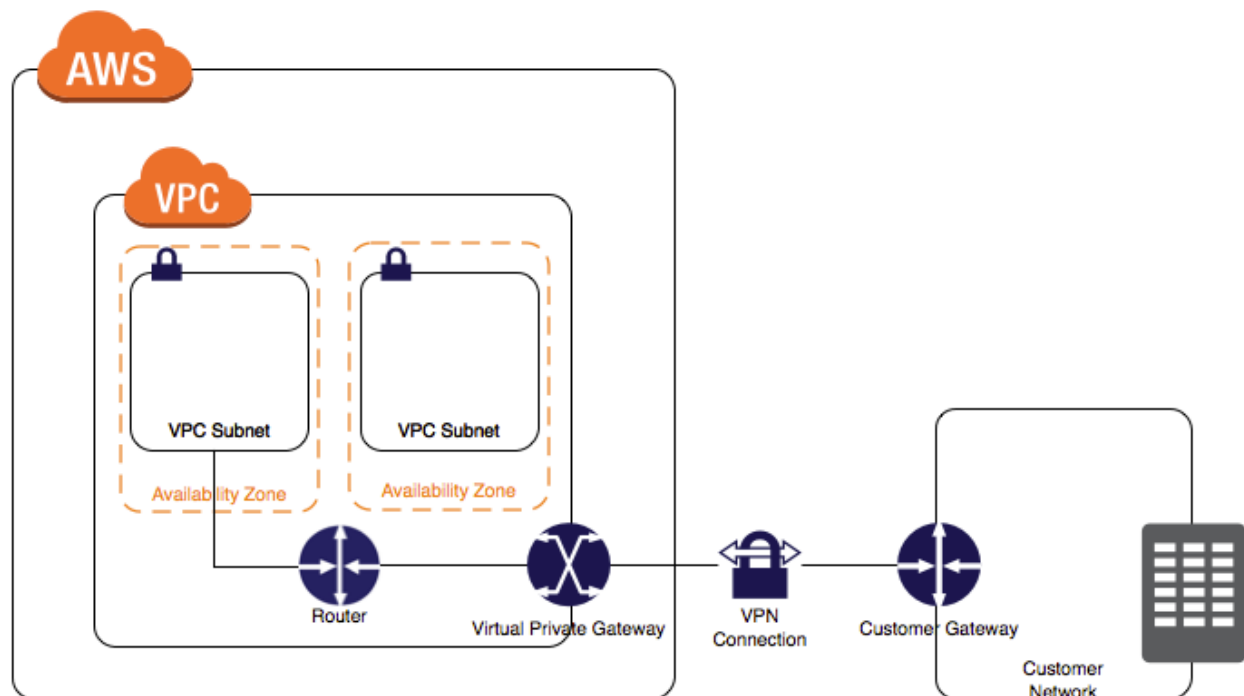
- When the server successfully decrypts the Ticket, it knows the enclosed data specifically the shared session key $K_{c,s}$ and the client's identity.
- The server then uses that newly extracted $K_{c,s}$ to decrypt the user's Authenticator.
- If the Authenticator successfully decrypts, and the identity inside matches, the server knows the request definitely came from the client. Furthermore, the server checks the timestamp. If it is fresh and falls within the Ticket's lifetime, the server authenticates the user and knows this is not a replay attack.

How the User Authenticates the Server:

The user knows that the true server is the only entity in the world capable of decrypting the Ticket to obtain the shared session key.

- After validating the Authenticator, the Server formulates a reply message containing the user's timestamp plus one and encrypts it using $K_{C,S}$.
- When the user receives this message, they decrypt it using their copy of $K_{C,S}$.
- Because the decrypted value successfully yields exactly $ts + 1$, it mathematically proves to the user that the server was able to read the Authenticator and possessed the correct session key.

Q4. AWS Site-to-Site VPN based on Diffie-Hellman Key Establishment



This diagram illustrates an AWS Site-to-Site VPN, which securely connects a Virtual Private Cloud (VPC) on the AWS side to an on-premises Customer Network.

Between VPC and Customer Network there is a VPN Connection using IPSec to encrypt all data exchanged. To establish this secure tunnel without having a pre-shared key sent over an insecure channel, the two gateways use the Diffie-Hellman key exchange method.

- (1) Implement 160-bit random number generator and secure hash algorithm SHA1;
- (2) Use a Crypto Library to implement the modular exponentiation algorithm for larger integers and use your implementation to output $y = g^x \pmod{p}$, where $x = \text{SHA1}(\text{your student ID})$.
- (3) After randomly generating 160-bit a and 160-bit b , output $(A, g^A \pmod{p})$ and $(B, g^B \pmod{p})$ and the secret key $g^{AB} \pmod{p}$ established between your VPC (named by your first name) and your data centre (named by your surname) by the Diffie-Hellman key exchange protocol, where $A = \text{SHA1}(a, \text{your first name})$ and $B = \text{SHA1}(b, \text{your surname})$.

Codes are implemented in the file `q4_diffie_hellman_key.py`

```

Modular Exponentiation
Student ID: S4129857
X (Hex): 8a3565db2be90db3a75f982e3fc1ccf7a9f9840b
Y = g^X mod p:
177136166648443069531467526512854650997959208150185272639346043590398338113461411603575
025702664753861203340557226478898470771099241400986176189039230237706555617966557581005
783541597954630353950351645030496004370642544462149644879672320272149806667202125136451
931734732904650114202914244749645541318167777155

Key Establishment
VPC (Anthony) outputs (A, g^A mod p):
(a91c163f0952bacad4bb8a299bdd99803ba008b4,
162170205040276026426543484431428526257704412711205041607038705208360222442208454015712
963351827957705453301724622331747577494870180434659546213813805087302128765123173280192
382529402763620934458110219866986336715196261311485936260024971246657611041476681115381
689568072684726200693435109465589654862458586885)

Data Centre (Fernando) outputs (B, g^B mod p):
(39009978cc78093f6c297c243efff06778e38af7,
117622104624396117571216797682823978764118023618199262323981489802405410717790283074917
989690503094856490402551844728429906393509324963290909554898222271833502843059954694344
384175910086374715173801771789819994905537030252487115235077602704624399263170281309951
9103671892249721714758469342482635209776359964)

Established Secret Key (g^AB mod p):
191865058386754670195622313731245817271049428779695671611342710644899540528895604888737
518209314422685659383192736235526790019048100923466470722272089232641241213702423056788
687394334775069116933145663293626104777488071821451862444523758608237009112177478053502
32038213742912825701979910344075722195367966035

```

(4) Can you perform a **Man-in-the-Middle Attack** to the Diffie-Hellman key exchange protocol? If so, show attacking steps.

Yes I can, the standard Diffie-Hellman key exchange protocol is highly vulnerable to a Man-in-the-Middle attack because it lacks authentication. Neither party can verify the true identity of the person they are communicating with over the insecure channel.

Attacking steps are like this,

- First user sends his public value $g^A \pmod p$ across the internet intended for second user. Attacker intercepts and drops this packet.
- Attacker generates own 160-bit random number C and computes public value $g^C \pmod p$. Attacker sends $g^C \pmod p$ to second user, spoofing IP to make it look like it came from First user.
- Second user computes public value $g^B \pmod p$ and sends it back. Attacker intercepts and drops this packet too.
- Attacker sends his public value $g^C \pmod p$ to first user, pretending it came from second user.
- First user computes the shared secret believing it's with the data centre: $K_1 = (g^C)^A \pmod p$. Second user computes the shared secret believing it's with the VPC: $K_2 = (g^C)^B \pmod p$.
- Attacker computes K_1 using her secret C against first user's intercepted g^A , and computes K_2 using secret C against second user's intercepted g^B .

(5) How does IKEv2 overcome the **Man-in-the-Middle Attack**? Show steps.

The Internet Key Exchange protocol (IKEv2) is used to establish a security association (SA) within the IPSec framework. IKEv2 defeats the Man-in-the-Middle attack by enforcing Authentication alongside the key exchange. It ensures that both the VPC and the Customer Gateway mathematically prove their identities before the shared secret is used to encrypt actual traffic.

Steps of IKEv2,

Both the VPC Gateway and the Customer Gateway possess a digital certificate (Ex: X.509) signed by a mutually trusted Certificate Authority (CA), which binds their identity to a public key.

When first user sends Diffie-Hellman public value to second user, IKEv2 requires the VPC to generate a digital signature over that message using its private RSA key.

Upon receiving the message, second user checks the digital signature against the public key listed in first user's verified X.509 certificate.

If attacker intercepts the message and tries to inject her own key, he must sign it. However, he does not possess first user's private RSA key, so he cannot generate a valid signature. Without a signature second user won't accept it.

Q5. SSL Handshake Protocol

(1) Choose your ephemeral public key with 1024 bits and set e as the largest prime factor of your student number.

In an SSL/TLS handshake utilizing ephemeral keys, the server does not use its long-term static public key to encrypt the Pre-Master Secret. Instead, it generates a temporary, single-use key pair for this session.

To create 1024-bit ephemeral public key, we must first determine specific public exponent, $e_{\text{ephemeral}}$. This must be the largest prime factor of student number 4129857.

```

-----BEGIN----- C:/Users/ishankumalasingh/Desktop/Assignment_1/q3_ssl_handshake.py -----
Ephemeral Key Parameters
e_ephemeral (Largest prime factor of 4129857): 281
n_ephemeral (1024-bit Modulus Hex):
6c6b70ab63391c54c5c5a40acf2488b6315e8d6cf06c2bf2c7bf70682791e56f00645af43fbd373d76608179142
8b8d6f16572f14a7f901973aaa74ab6a24aa70c4909aaa24cb31cc6c2204f9c05f7ca2e2b02bc0b3010fd0b1510
37bedebb15ec7a21d39b95a35c015f4c38bfa107d88e0f853f2d247feed51f5976c7f5bbc1

ServerKeyExchange Signature
Signature (Hex):
7c45f1edc2767505cda4e0fd1a0eb2c836a110ebbc6c6e8802588c1f723296970394d3528f6379259b5e2f6d6d1
4fe923e2e2a5040ba73d2380525a40ad06f41a9bb63e7e6b66f14c272f37f1b7f9bbbc2c71732bd2c38725e1699
37aa1888d21edf28dc0fe651989906e672ef3d160584b3740cb2ac2099582409f0380135b13402dad64be8da95a
860de80db443bf44a5b9c2cf9975a11de174198cbde599b87e8778f39e781418a964e7c15db39624f73198b96f
7f553f4e4cbcb88e7bf1f7e000d49d4d7d2227da376f9672b091c6e893ae524f64b8c8f01da84c5e6c9e6b5d99
d5722bd9b151b88e7f89e262a4a131d0ae77208642420582c75a6f0a5

ClientKeyExchange Message
Pre-Master Secret (Hex): 68460c9658343c7dbd3ef4e6ae35838fe3224121ca7d056809fe15d8d5518053ea
6df46c199d5fb42de03c7b090ec5fc
Encrypted Pre-Master Secret (Hex):
5033d98178f9233c1e5148b0a11fe48db310022cc8d620608347398193d3d5635bbb9977196beb927632a489fac
7bedfa4c8ab8e20218148589185ae0f4457653e00885f8fa053c37088dd3c9aa84932a8ffc93f329b74a0a07158
40b1cf24b7b2ffa1f7aba3b57f7b85c396fdb6f227d016b55a8fd8d75d2e2a35bb75213926

```

Through prime factorization, we get: **281**

Since 281 is a prime number, it serves as ephemeral exponent **e**. Next, we generate a random 1024-bit RSA modulus, $n_{\text{ephemeral}}$. The combination of $(n_{\text{ephemeral}}, e_{\text{ephemeral}})$ constitutes ephemeral public key.

(2) What is the ServerKeyExchange message (Hex numbers) in Figure 5?

ServerKeyExchange message is sent immediately after the Certificate message. Its purpose is to securely transmit the ephemeral public key to the client.

To prove that this ephemeral key belongs to the server and hasn't been injected by a Man-in-the-Middle attacker, the server creates a digital signature. It hashes the ephemeral public key parameters and signs that hash using its long-term private key d , which corresponds to the public key in its X.509 certificate.

Serverkey Exchange Signature -

```

7c45f1edc2767505cda4e0fd1a0eb2c836a110ebbc6c6e8802588c1f723296970394d3528f6379259
b5e2f6d6d14fe923e2e2a5040ba73d2380525a40ad06f41a9bb63e7e6b66f14c272f37f1b7f9bbbc2c
71732bd2c38725e169937aa1888d21edf28dc0fe651989906e672ef3d160584b3740cb2ac2099582
409f0380135b13402dad64be8da95a860de80db443bf44a5b9c2cf9975a11de174198cbde599b87
e8778f39e781418a964e7c15db39624f73198b96f7f553f4e4cbcb88e7bf1f7e000d49d4d7d2227da
376f9672b091c6e893ae524f64b8c8f01da84c5e6c9e6b5d99d5722bd9b151b88e7f89e262a4a131d
0ae77208642420582c75a6f0a5

```

- (3) If Pre_Master_Secret is SHA384(your real email address), where the hash function is SHA384 (<https://emn178.github.io/online-tools/sha384.html>), what is the ClientKeyExchange message (Hex number) in Figure 5?

Once the client receives the ServerKeyExchange message and verifies the signature, it generates a random Pre-Master Secret. Pre-Master Secret is derived by hashing email address using SHA-384.

The client must send this Pre-Master Secret to the server without anyone else intercepting it. It achieves this by encrypting the Pre-Master Secret using the server's ephemeral public key.

The ClientKeyExchange message will contain the resulting cipher text:

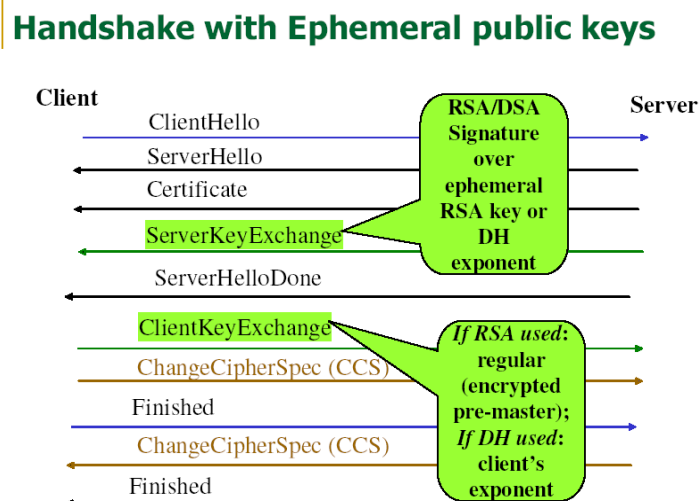
Pre-Master Secret (Hex):

68460c9658343c7dbd3ef4e6ae35838fe3224121ca7d056809fe15d8d5518053ea6df46c199d5fb42de03c7b090ec5fc

Encrypted Pre-Master Secret (Hex):

5033d98178f9233c1e5148b0a11fe48db310022cc8d620608347398193d3d5635bbb9977196beb927632a489fac7bedfa4c8ab8e20218148589185ae0f4457653e00885f8fa053c37088dd3c9aa84932a8ffc93f329b74a0a0715840b1cf24b7b2ffa1f7aba3b57f7b85c396fdb6f227d016b55a8fd8d75d2e2a35bb75213926

- (4) Analyse client authentication and server authentication of the handshake protocol.



Client Authentication- In the specific handshake flow showed in above figure, there is no client authentication. The server does not send a CertificateRequest, and the client doesn't send a ClientCertificate or CertificateVerify message. The client remains anonymous to the server at the TLS layer.

Server Authentication- The protocol successfully achieves server authentication. When the client receives the ServerKeyExchange message, it extracts the signature and verifies it using the server's long-term public key (e, n) found in the server's certificate. Because only the true server possesses

the corresponding long-term private key d , a valid signature guarantees that the client is communicating with the legitimate server.

(5) Analyse the forward security of the handshake protocol.

The handshake protocol depicted in above figure provides Perfect Forward Secrecy (PFS).

If the server use its static, long-term RSA key to encrypt the Pre-Master Secret directly, an attacker who records the encrypted traffic could hold onto it for years. If that attacker ever compromises the server's long-term private key d in the future, they could retroactively decrypt the Pre-Master Secret and read all historical communications.

By using an ephemeral key exchange, the Pre-Master Secret is encrypted with a temporary key. Once the session is over, the ephemeral private key is destroyed. Even if an attacker compromises the server's long-term private key d later, they only gain the ability to spoof the server in future sessions and they cannot decrypt past sessions because the ephemeral key needed for decryption no longer exists.